

Relatório: Lista 3 - Matchmaking

Ana Julia da Silva e Lisânia Soares Miranda

Maio 2026

1 Descrição do Projeto

Este trabalho implementa um sistema de *matchmaking* para jogos online competitivos, em que jogadores aguardam em uma fila e são agrupados de acordo com a proximidade de seus níveis de habilidade (*score*). O sistema é implementado em C++ por meio de duas classes principais: **Player**, que encapsula os atributos de cada jogador (identificador, nome, *score* e *timestamp* de chegada), e **Matchmaking**, responsável por gerenciar o array estático de jogadores em espera.

A ideia central do projeto é utilizar ordenação para viabilizar a formação eficiente de grupos. Ao manter os jogadores ordenados por *score*, é possível encontrar o primeiro grupo válido com uma simples varredura linear, em vez de avaliar todas as combinações possíveis. O sistema oferece duas estratégias de ordenação, *insertion sort* e *merge sort*, implementadas manualmente, sem uso de funções prontas da STL, o que permite comparar empiricamente o desempenho de um algoritmo $O(n^2)$ frente a um $O(n \log n)$ em cenários com volumes crescentes de jogadores.

2 Implementação dos Algoritmos de Ordenação

O sistema implementa dois algoritmos de ordenação sobre o array interno de jogadores, ordenando por *score* crescente e, em caso de empate, por *timestamp* crescente.

2.1 Insertion Sort

O *insertion sort* percorre o array a partir do segundo elemento. Para cada elemento na posição e , compara-o com os elementos já ordenados (posições 0 a $e - 1$). Quando encontra a posição correta, isto é, o primeiro índice i tal que `players[i].score > current.score` (ou empate resolvido por *timestamp*), o método auxiliar `move_player` desloca todos os elementos entre i e e uma posição à direita e insere o elemento corrente em i .

A decisão de encapsular o deslocamento em `move_player` mantém o laço externo legível e separa a responsabilidade de reposicionamento da lógica de ordenação.

2.2 Merge Sort

O *merge sort* foi implementado recursivamente por meio de dois métodos auxiliares: `merge_sort(left, right, aux)` e `merge(left, mid, right, aux)`.

`merge_sort` divide o subarray ao meio, chama a si mesmo para ordenar cada metade e depois invoca `merge` para combinar as duas partes já ordenadas. O caso base ocorre quando `left >= right`, ou seja, o subarray possui zero ou um elemento.

`merge` utiliza dois cursores (i para a metade esquerda, j para a direita) e copia o menor elemento a cada passo para um array auxiliar de tamanho `MAX_PLAYERS`, alocado uma única vez em `sortByScoreMerge` e passado por ponteiro para evitar realocações repetidas. Ao final, o trecho ordenado é copiado de volta para `players`.

3 Uso da Ordenação na Formação de Grupos

O método `formGroup(groupSize, delta, n)` pressupõe que o array já está ordenado por *score*. Com essa garantia, a busca pelo primeiro grupo válido reduz-se a uma varredura linear com uma janela deslizante de tamanho `groupSize`:

- Para cada posição inicial i , compara-se `players[i + groupSize - 1].score` com `players[i].score`.
- Como o array está ordenado, o elemento em i possui o menor *score* da janela e o elemento em $i + groupSize - 1$ possui o maior. Portanto, a diferença `maior_score - menor_score` pode ser verificada com custo $O(1)$ por janela.
- Ao encontrar um grupo válido, os jogadores são copiados para um array alocado dinamicamente, removidos do array interno por deslocamento à esquerda, e, por fim, o método retorna o grupo formado imediatamente.

Sem a ordenação prévia, seria necessário avaliar todas as combinações de `groupSize` jogadores dentre os n disponíveis, resultando em custo $O\left(\binom{n}{\text{groupSize}}\right)$. Com o array ordenado, a varredura linear tem custo $O(n)$.

4 Custo Computacional das Operações

Operação	Complexidade	Justificativa
Inserção	$O(1)$	Inserção no final do array.
Remoção	$O(n)$	Busca linear pelo ID; deslocamento dos elementos.
Insertion sort	$O(n^2)$	Dois laços aninhados no pior caso.
Merge sort	$O(n \log n)$	Divisão e conquista com $\log n$ níveis de recursão.
Formação de grupo	$O(n)$	Varredura linear com janela de tamanho fixo.
Recuperação (<code>getWaitingPlayers</code>)	$O(n)$	Cópia integral do array interno.

5 Comparação Experimental entre Insertion Sort e Merge Sort

5.1 Metodologia

Os testes foram realizados em um arquivo `benchmark.cpp` separado do `main.cpp`, de forma a não interferir nos casos funcionais. Para cada tamanho de entrada $n \in \{100, 500, 1000, 5000, 10000, 50000, 100000\}$, foi gerado um array de jogadores com *scores* aleatórios no intervalo $[0, 10^6]$ e *timestamps* sequenciais. O mesmo array foi copiado para as duas instâncias de `Matchmaking` antes de cada medição, garantindo entradas equivalentes. O tempo foi medido com `std::chrono::high_resolution_clock` e cada configuração foi executada uma única vez. Os resultados foram exportados para `benchmark.csv` e os gráficos gerados pelo script `graphic_comparison.py`.

5.2 Resultados

Número de jogadores	Insertion Sort (ms)	Merge Sort (ms)
50	0,0458	9,1529
100	0,2746	4,9749
500	1,5365	3,5624
1 000	9,3795	7,315
2 000	31,037	7,2209
5 000	158,707	5,5085
10 000	922,731	11,1351
20 000	4 707,26	16,4859
50 000	42 262,8	262,813
100 000	156 851	90,4022

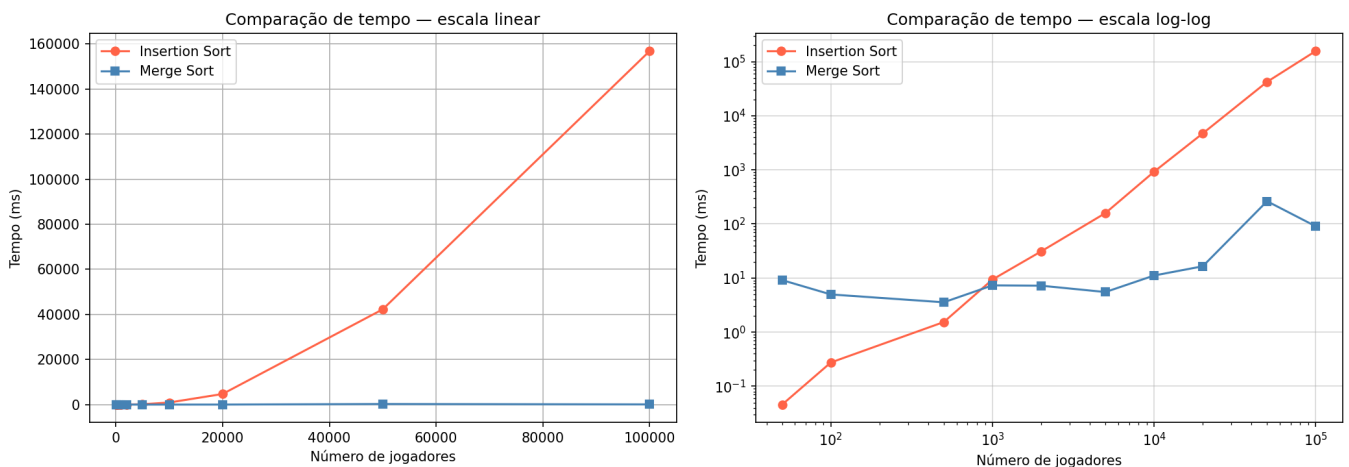


Figure 1: Comparação de tempo de execução entre Insertion Sort e Merge Sort em escala linear (esquerda) e log-log (direita).

5.3 Análise

Os resultados estão coerentes com as complexidades teóricas. O *insertion sort* cresce quadraticamente: ao dobrar n de 50 000 para 100 000, o tempo aproximadamente quadruplica (de $\approx 1,1$ s para $\approx 4,5$ s), comportamento típico de $O(n^2)$. O *merge sort* cresce de forma muito mais suave, próxima de $O(n \log n)$: ao passar de 50 000 para 100 000 jogadores, o tempo pouco mais que dobra (de $\approx 4,2$ ms para $\approx 9,0$ ms).

Para entradas pequenas ($n \leq 1\,000$), ambos os algoritmos são praticamente instantâneos e a diferença é desprezível na prática. A partir de $n = 5\,000$, a vantagem do *merge sort* torna-se evidente, e para $n = 100\,000$ (capacidade máxima do sistema) a diferença é de cerca de **500**×, tornando o *insertion sort* inviável em cenários com muitos jogadores simultâneos.

Em um sistema de matchmaking real, onde novas entradas e saídas ocorrem continuamente, o *merge sort* é claramente preferível para reordenações completas do array. O *insertion sort*, porém, mantém utilidade teórica em arrays quase ordenados — situação que pode ocorrer quando apenas um ou poucos jogadores são inseridos após uma ordenação prévia — onde seu custo tende a $O(n)$.